

Home

Logged Out

Flasky

Home

Log In

Hello, Stranger !

Pleased to meet you!

All



Brady Finnell

a month ago

Building a Secure Flask API with OAuth 2.0 and Token Authentication

Over the past few weeks I've been integrating **OAuth 2.0** and **HTTPTokenAuth** into a Flask application, and the experience taught me more about web security than any textbook could. This post breaks down the full architecture — what works, what doesn't, and the specific gotchas that cost me hours of debugging.

The Two-Layer Auth Model

The biggest conceptual shift was realizing that *browser authentication* and *API authentication* are completely separate concerns. Google OAuth handles your human users sitting at a browser. HTTPTokenAuth handles programmatic API clients. They coexist in the same app without stepping on each other.

- **Layer 1 — Browser:** User clicks "Login with Google," gets redirected, consents, and lands back with a Flask session cookie
- **Layer 2 — API:** That same logged-in user POSTs to `/api/v1/tokens` to exchange their session for a bearer token
- **Layer 3 — Token use:** API clients include `Authorization: Bearer <token>` on every subsequent request

Home Feed (HTML & Plain text)



sarah_writes

a month ago

I spent way too long being afraid of Flask-Migrate. The commands looked cryptic and I had broken my database schema more than once trying to figure it out. Here's what finally clicked for me: a migration is just a Python script that describes what changed between your old schema and your new one. Flask-Migrate uses Alembic under the hood to auto-generate these scripts by comparing your SQLAlchemy models against the current database state. The workflow is simple: change your model, run flask db migrate to generate the script, review the generated file to make sure it looks right, then run flask db upgrade to apply it. The key habit I developed was always reviewing the generated migration before running upgrade. Alembic doesn't always detect every change correctly, especially with constraints and indexes.

Permalink

2 Comments



jharrison

a month ago

This weekend I built a small REST API using Flask and SQLAlchemy and learned a ton in the process. My goal was to expose some of my local data through a clean JSON API with proper authentication.

The hardest part wasn't the routes — it was getting the token authentication right. I ended up using `flask-httpauth` with a bearer token scheme and `itsdangerous` for generating time-limited tokens. Here's the basic token generation:

```
def generate_auth_token(self, expiration=3600):
    s = Serializer(SECRET_KEY)
    return s.dumps({'id': self.id})
```

The thing I kept forgetting is that `s.loads()` needs the `max_age` argument to actually enforce expiration. Without it, the token lives forever regardless of what you passed to `dumps()`.

Permalink

2 Comments



mwilliams

a month ago

Six months ago I made a decision that felt risky at the time: I dropped React from a small internal tool project and rebuilt the frontend with HTMX. Here's where I stand now. The development speed difference is real and significant. Without a build step, without managing component state, without npm, I'm shipping features in a fraction of the time. HTMX lets the server stay in charge of rendering, which means my Flask templates do the heavy lifting and the browser just swaps HTML fragments in and out. For a data-heavy CRUD app, this is honestly the right call. I still love React for complex interactive UIs. But for most of what I was actually building — forms, tables, dashboards — HTMX is faster to write, easier to debug, and way simpler to deploy.

Permalink

2 Comments

Navbar

Logged Out

Advancing AI Feed Explore Tags

Log In Register

Logged In

Advancing AI Feed Explore Tags My Profile Create Post

bradyfinnell ▾

OAuth 2.0 Login



Advancing AI Newsletter wants to access
your Google Account

 @gmail.com

This will allow Advancing AI Newsletter to:

-  See your personal info, including any personal info you've made publicly available 
-  See your primary Google Account email address 
-  Associate you with your personal info on Google 

By clicking Allow, you allow this app and Google to use your information in accordance with their respective terms of service and privacy policies. You can change this and other [Account Permissions](#) at any time.

Deny

Allow

Profile Page

[Flasky](#)[Home](#)[Log In](#)

tech_raj

Raj Patel From [San Jose, CA](#)

ML engineer building cool things with Python and TensorFlow.

Member since 06/20/2026.

Last seen a month ago.

Followers: **1** Following: **1**

Posts by tech_raj

[tech_raj](#)

a month ago

Flask blueprints are one of the most powerful features for structuring large applications. When your app grows beyond a few routes, throwing everything into a single file becomes a nightmare to maintain. Blueprints let you split your application into smaller, self-contained modules — each with its own routes, templates, and static files. Think of them like mini-apps that get assembled together by the application factory. You register a blueprint with a URL prefix, so all its routes live under a shared namespace like /auth or /api. This keeps your code organized, testable, and easy to reason about. If you've been putting off refactoring your monolithic Flask app, start with blueprints.

[Permalink](#)[2 Comments](#)

Explore Post Tags

Advancing AI Feed Explore Tags My Profile Create Post

bradyfinnell ▾

Posts tagged " flask"

[Testing for HTML Rendering](#)

Building a Secure Flask API with OAuth 2.0 and Token Authentication

Over the past few weeks I've been integrating **OAuth 2.0** and **HTTPTokenAuth** into a Flask application, and the experience taught me more about web security than any textbook could. This post breaks down the full architecture — what works, what doesn't, and the specific gotchas that cost me hours of debugging.

The Two-Layer Auth Model

The biggest conceptual shift was realizing that *browser authentication* and *API authentication* are completely separate concerns. Google OAuth handles your human users sitting at a browser. HTTPTokenAuth handles programmatic API clients. They coexist in the same app without stepping on each other.

- **Layer 1 — Browser:** User clicks "Login with Google," gets redirected, consents, and lands back with a Flask session cookie
- **Layer 2 — API:** That same logged-in user POSTs to `/api/v1/tokens` to exchange their session for a bearer token
- **Layer 3 — Token use:** API clients include `Authorization: Bearer <token>` on every subsequent request

Token Expiration — The Detail That Bites Everyone

The book's example uses `itsdangerous` for token generation, but there's a subtle trap. Look at these two lines:

```
# BROKEN – token never actually expires
```

Tags

caching	1
CSS	2
flask	2
HTML	2
LLM	2
LLMs	1
testing	2

Example Profile – Admin Edit

Advancing AI

Feed

Explore Tags

My Profile

Create Post

bradyfinnell

Edit Profile

ADMIN ONLY - SENSITIVE INFORMATION.

Email

raj.patel@email.com

Role

Editor

END ADMIN DATA

Real name

Raj Patel

Username

tech_raj

Location

San Jose, CA

About me

ML engineer building cool things with Python and TensorFlow.

age

27

Submit